ACBS v2: Project Pages Blueprint

Auto Classifier Benchmarking System with 9 AutoML frameworks. How the finished product should look, how many pages it needs, how each page works, and a ready prompt for every page. Example numbers inside wireframes are placeholders, not results.

1. How many pages? (Answer)

9 pages in total: 8 in the sidebar menu and 1 drill-down page reached from Results. A senior developer resists adding pages. There is no login page (single-user local tool), no separate Export page (export is a button), and no admin page (settings live inside Frameworks). Every page below answers one question the user actually asks.

#	Page	Question it answers	In menu?	Priority
1	Home	What is the state of my work and what should I do next?	Yes	P3 nice to have
2	Datasets	What is in my data and is it safe to benchmark?	Yes	P1 must
3	New Benchmark	What exactly do I want to compare, with which budget and priorities?	Yes	P1 must
4	Run Monitor	Is it running, which framework is it on, did anything fail?	Yes	P1 must
5	Results	Which model or framework should I use, and why?	Yes	P1 must
6	Model Detail	How good is this one candidate, in depth?	No (opens from Results)	P2 should
7	History	What did I run before, and how do two runs differ?	Yes	P2 should
8	Frameworks	Which AutoML tools work on this machine, and with what settings?	Yes	P2 should
9	Methodology	How are the numbers produced, and can I trust them?	Yes	P3 nice to have

Priority meaning: P1 pages give you a complete demo (upload, configure, run, see a recommendation). Build P1 first, then P2, then P3. If time runs out, P3 can be cut without breaking the product.

2. Look and Feel (applies to every page)

Direction: a calm, data-first research tool. Think lab notebook crossed with a clean analytics workbench, not a flashy SaaS landing page. Whitespace and clear hierarchy do the work; there are no gradients, stock illustrations or decorative animations.

Principles

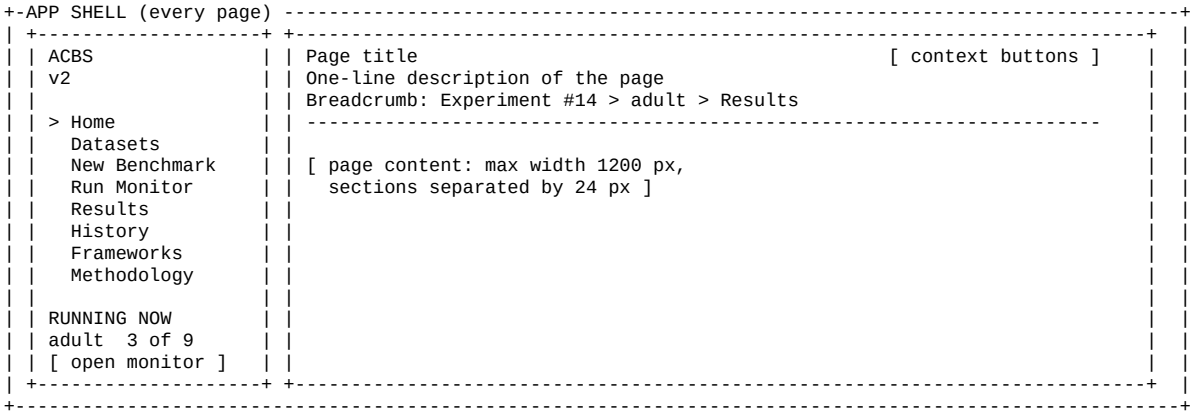
- **The answer comes first.** Results opens with the recommendation, then the evidence underneath it.
- **Never hide cost.** Time, budget used and memory sit next to accuracy everywhere, because efficiency is the novelty of this project.

- **Be honest about failure.** Failed, timed-out and unavailable frameworks are shown as rows with reasons, never dropped.
- **Two tiers, always distinguishable.** Classic baselines and AutoML frameworks are told apart by a text badge and a marker shape, not by colour alone.
- **Explain, don't just display.** Each chart has a one-line caption saying what it shows and what to look for.

Visual system

Element	Rule
Frontend stack	Plain HTML, CSS and JavaScript (ES modules), no framework and no build step. 9 HTML files in web/, served by FastAPI at the same origin; JSON API under /api; Plotly.js vendored locally; strict Content-Security-Policy; Playwright for end-to-end tests.
Theme	Light by default using CSS custom properties in web/css/tokens.css; all colours, spacing and type sizes defined once as tokens so a dark theme only overrides the tokens.
Typography	One sans-serif family (Inter or system default). Page title 28 px, section 20 px, body 15 px, table/caption 13 px. Numbers use tabular figures so columns align.
Colour roles	One accent (deep teal) for primary actions and the recommended model. Neutral greys for structure. Status colours only for status: green ok, red failed, amber timeout, grey unavailable/unsupported. Each status also has an icon and a word.
Tier markers	Classic = circle marker + blue-grey badge "Classic". AutoML = diamond marker + orange badge "AutoML". Used identically in tables and charts.
Spacing	8 px base grid. Page max width about 1200 px, centred. 24 px between sections, 16 px inside cards.
Charts	Plotly.js (vendored) with one shared layout (same fonts, gridlines, colours). Log scale on time axes with the axis labelled "log". Hover shows exact values.
Motion	None except progress bars and spinners. No animated transitions.

3. Layout and Navigation



- **Sidebar** (shared nav built by web/js/shell.js on every page) lists the 8 menu pages in workflow order. The active page is highlighted and also marked with text, not only colour.
- **Active-run chip** at the bottom of the sidebar appears only while a benchmark is running and links to the Run Monitor from any page.

- **Header** shows the page title, one sentence of help, and a breadcrumb when an experiment is open.
- **Model Detail** is not in the menu; it opens from a row in Results and has a back link.
- **Responsive:** designed desktop-first. At about 1024 px columns stack and tables scroll horizontally. Phone layout is not a goal (this is a benchmarking workbench); say so in the README.
- **Accessibility:** text contrast at least 4.5:1, every status has text plus icon, all controls reachable by keyboard, every chart has a caption and a data-table fallback.

4. Shared Components (build these first)

These are reused on every page. Building them once keeps the app consistent.

Component	What it shows	Used on
Tier badge	Classic or AutoML pill	Results, Model Detail, Run Monitor, History
Status chip	ok / failed / timeout / running / queued / skipped / unavailable / unsupported with icon and word	Run Monitor, Results, Frameworks
Metric tile	Label, large value, small caption (for example "F1 macro", 0.87, "mean of 5 folds")	Home, Datasets, Model Detail
Recommendation card	Winner, explanation text, stability %, AutoML verdict, caveats, action buttons	Results, History compare
Leaderboard table	Sortable ranked table with tier badges and greyed excluded rows with reason	Results, Run Monitor
Chart frame	Plotly figure plus a one-line caption and a "show data" toggle	Results, Model Detail
Cost banner	Estimated wall-clock time with the reason it is long	New Benchmark
Empty state	What this page is for, why it is empty, and one button to fix that	All pages
Error banner	What failed, what it means, what to do next, retry button	All pages
Priority panel	Preset dropdown, weight sliders, constraints	New Benchmark, Results

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint, ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Create the design foundation and the shared components for the HTML/CSS/JS frontend, before any page is built.

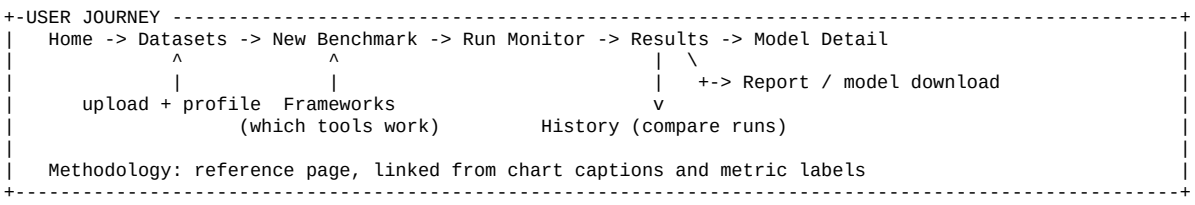
Requirements:
- Create web/css/tokens.css (theme tokens from Section 2), base.css (reset, typography, layout grid, tabular numbers) and components.css (badges, status chips, tiles, tables, banners). No framework and no CDN: vendor any library locally under web/vendor/.

- Create web/js/components/ (one ES module each) with: tier_badge, status_chip, metric_tile, recommendation_card, leaderboard_table, chart_frame, cost_banner, empty_state, error_banner, priority_panel. Each takes plain data (objects or arrays) and returns a DOM element built with createElement and textContent.
- Create web/js/charts.js: vendor Plotly.js locally and define one shared layout/config (fonts, gridlines, colours, tier marker shapes: circle for Classic, diamond for AutoML). The backend builds figure JSON from src/acbs/reporting/charts.py and serves it from /api; the frontend only renders it with the shared layout.
- Create web/js/api.js as the only module that calls fetch, with timeouts, AbortController support, one error shape and an optional API token read from sessionStorage and sent as an Authorization header. Move the existing API routes under /api and serve web/ as static files from FastAPI at the same origin (report every route moved).
- Create the 9 HTML files in web/ (index, datasets, new-benchmark, run-monitor, results, model-detail, history, frameworks, methodology) sharing one shell (sidebar in Section 1 order, header, active-run chip, toast area) with placeholder content, and add a strict Content-Security-Policy that forbids inline scripts.
- Add a throwaway web/_gallery.html that renders every component in every state so they can be checked visually.

RULES:
Follow the visual system exactly. Colour is never the only signal. Do not build real page content yet. The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement, run the app, and report files created, routes moved, the Plotly.js version vendored, and any browser limitation worked around.

5. Page Flow

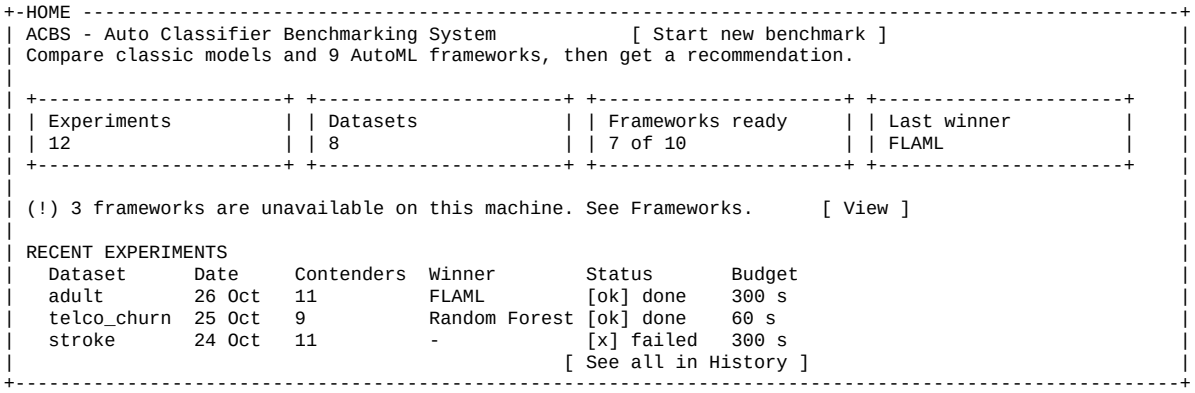


Main path (about 5 clicks): Datasets (pick one) > New Benchmark (choose contenders, budget, priorities) > Run Monitor (wait) > Results (read recommendation) > Model Detail or Download. A first-time user can also click "Try a sample dataset" on Home and reach Results in one guided run.

6. Page 1: Home

Purpose: orient the user, show what is running or ready, and give one obvious next action. **Priority P3.**

How it looks



How it works

- Tiles read from GET /experiments, /datasets and /frameworks. A tile is a link to its page.
- A running experiment replaces the top banner with a progress card (dataset, framework 3 of 9, elapsed) and an "Open monitor" button.
- Clicking a recent-experiment row opens its Results page. Failed rows open the Run Monitor so the reasons are visible.
- The "unavailable frameworks" banner shows only if at least one framework cannot run, and it links to Frameworks.

States

- **First run (empty):** three-step guide (choose data, choose contenders, read the recommendation) and a button "Try a sample dataset (small, about 2 minutes, classic models only)".
- **Loading:** skeleton tiles. **API down:** error banner with Retry and the command to start the API.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the Home page (web/index.html and web/js/pages/home.js).

Requirements:
- Four metric tiles, active-run card (only when a run exists), unavailable-frameworks banner, recent experiments table (5 rows), primary button to New Benchmark.
- First-run empty state with the three-step guide and a sample-dataset button that creates a classic-only benchmark on a small built-in dataset and opens the Run Monitor.
- Error and loading states as described in the blueprint.

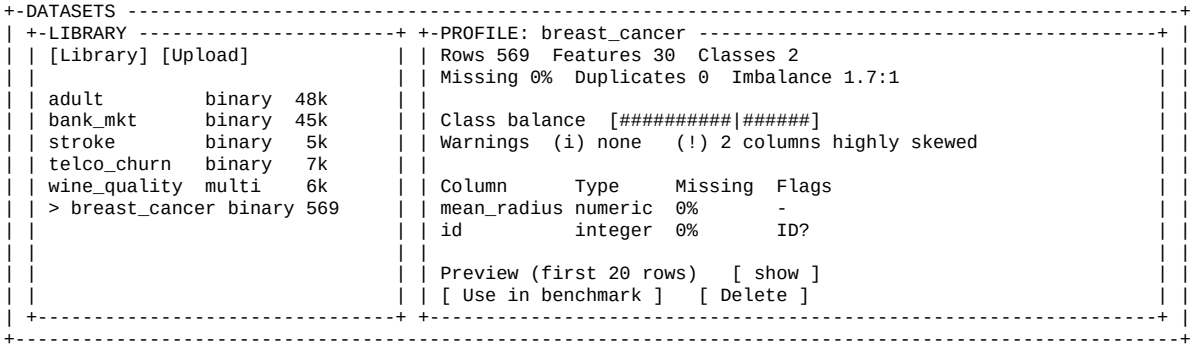
RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement and run the page. Report files changed, endpoints used, states verified and anything left out.

7. Page 2: Datasets

Purpose: manage datasets and show a trustworthy profile before any benchmark starts. This is where bad data is caught. **Priority P1.**

How it looks



How it works

- **Library tab:** built-in registry datasets plus uploaded ones. Selecting a row loads its profile on the right (GET /datasets/{id}).
- **Upload tab:** drag and drop CSV, Excel or Parquet (size limit shown). After upload the user picks the target column (last column suggested) and the profile appears.
- **Validation rules shown as plain messages:** target missing; fewer than 2 classes; fewer than 50 rows; a numeric target with more than 50 unique values is rejected as regression; all-null or duplicate column names flagged.
- **Profile warnings** carry a severity (info, warning, blocking) with text, for example "Severe class imbalance 1:577; accuracy will be misleading, macro-F1 will be emphasised".
- **Use in benchmark** carries the dataset and target to New Benchmark.

States

- **Empty library:** button to download the built-in datasets. **Uploading:** spinner with file name. **Rejected file:** error banner with the exact rule broken and how to fix it. **Delete:** confirmation dialog that says which experiments reference the dataset.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the Datasets page (web/datasets.html and web/js/pages/datasets.js) with Library and Upload tabs and the profile panel.

Requirements:
- Library table with filters by domain and task; selecting a row shows the profile panel: tiles, class balance bar, column table with flags (ID, constant, high cardinality, high missing), warnings with severity, and a 20-row preview.
- Upload tab with file size limit from settings, target column picker, and the validation messages listed in the blueprint, using the backend validators (do not duplicate rules in the UI).
- Delete with a confirmation dialog listing dependent experiments.
- Button that stores dataset and target in session state and navigates to New Benchmark.

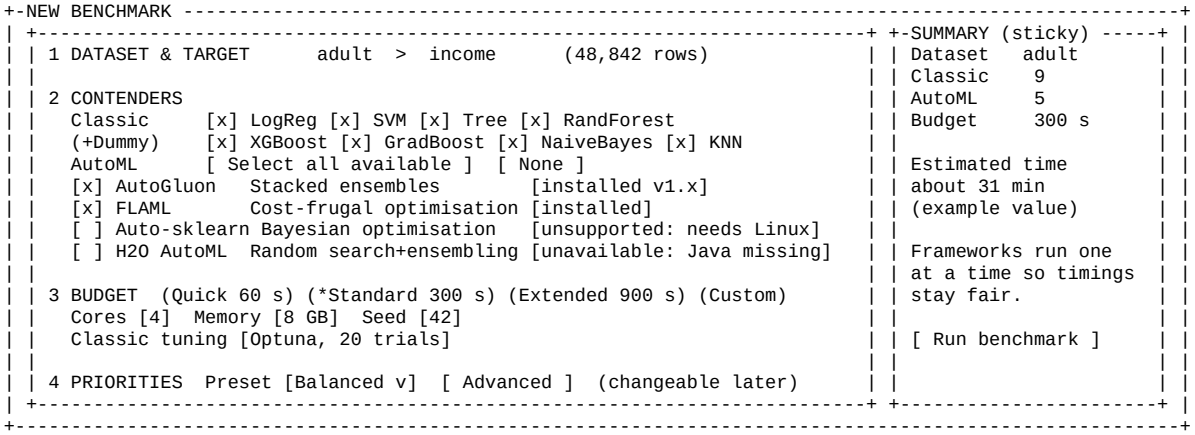
RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement and test with a valid file, a missing-target file, a single-class file and a 10-row file. Report files changed, tests run and issues.

8. Page 3: New Benchmark

Purpose: define the experiment in one place: data, contenders, budget, priorities, then run. The heart of the control flow. **Priority P1.**

How it looks



How it works

- **Four sections on one page** with a sticky summary on the right (a wizard would hide the cost; one page lets the user see everything change the estimate).
- **Contenders:** each AutoML card shows name, one-line search strategy, and an availability badge with the reason. Unavailable or unsupported frameworks are disabled but visible. Auto-sklearn and Auto-sklearn 2 are two cards (the backend counts 10 adapters for the 9 frameworks).
- **Budget:** presets plus custom. Each change calls POST /estimate and updates the cost banner. The banner explains why the total is the sum of all budgets (sequential runs).
- **Smart warnings:** imbalanced dataset suggests macro-F1 as the optimisation target; very large data warns that SVM and KNN may be skipped.
- **Run:** disabled until a dataset and at least one contender are chosen. If the estimate exceeds the configured limit (default 60 minutes), a confirmation dialog appears. Success opens the Run Monitor.
- **Priorities** use the shared Priority panel; they only affect ranking, so the user may change them after the run.

States

- **No dataset:** section 1 shows an empty state linking to Datasets. **No contender selected:** Run disabled with the reason beside it. **Estimate failed:** estimate hidden, run still allowed with a warning.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the New Benchmark page (web/new-benchmark.html and web/js/pages/new-benchmark.js).

Requirements:
- Four sections plus a sticky summary: dataset and target; contenders (8 classic checkboxes with Dummy locked on, AutoML framework cards from GET /frameworks with badges and reasons, select all available / none); budget and resources (Quick 60 s, Standard 300 s, Extended 900 s, Custom, cores, memory, seed, classic tuning); priorities via the shared Priority panel.
- Live cost estimate through POST /estimate with debouncing, shown in the cost banner and the summary.
- Smart warnings from the dataset profile (imbalance, size).
- Run button with disabled reasons, confirmation dialog above the configured time limit, then POST /experiments and navigate to Run Monitor.
- Persist the form state in sessionStorage so navigating away and back does not lose choices.

RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement and test: no dataset, no contender, unavailable framework disabled, long-run confirmation, successful start. Report files changed and endpoints used or added.

9. Page 4: Run Monitor

Purpose: show live progress of a long run (minutes to hours) and make failures understandable. **Priority P1.**

How it looks

```
+-RUN MONITOR ------------------------------------------------------------------------------------------+
| Experiment #14  adult > income        [running]   Elapsed 12:40   Left about 18 min   [ Cancel ]       |
| Overall  [##########--------------------]  4 of 11 contenders                                          |
|                                                                                                        |
| Contender        Tier     State          Time / Budget        Best so far     Log                    |
| Dummy baseline   Classic  [ok] done      0.1 s                                                         |
| Random Forest    Classic  [ok] done      14 s                                                          |
| AutoGluon        AutoML   [ok] done      301 s / 300 s        -               [view]                   |
| FLAML            AutoML   [..] running   96 s / 300 s  [###-------]  F1 0.81   [view]                 |
| H2O AutoML       AutoML   [-] queued                                                                   |
| Auto-sklearn     AutoML   [x] unsupported  needs Linux (WSL2/Docker)                                   |
|                                                                                                        |
| v Live log (last 50 lines)                                                                             |
|   [FLAML] iter 31  best learner lgbm  ...                                                              |
| When finished:  [ View results ]                                                                       |
+--------------------------------------------------------------------------------------------------------+
```

How it works

- Polls GET /experiments/{id}/status every 2 seconds with a JavaScript polling loop that updates only the table rows in place, not the whole page; it pauses while the browser tab is hidden.
- State is stored in the database, so closing the tab, refreshing, or reopening from History resumes the live view. If the backend restarted, the run shows **interrupted** with an option to rerun.
- **Time / Budget bar** turns amber past 100% (framework overrun, expected) and the state becomes timeout if the hard limit passes.
- **Failed** rows expand to show the reason and the last log lines (never secrets or full paths).
- **Cancel** asks for confirmation, stops the current framework process, keeps results gathered so far, and marks the run cancelled.
- **When done:** a green banner and a View results button; partial results are viewable earlier through a link.

States

- **No active experiment:** empty state with links to History and New Benchmark. **Lost connection:** banner "Reconnecting..." that keeps the last known state visible.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the Run Monitor page (web/run-monitor.html and web/js/pages/run-monitor.js) for live and finished experiments.

Requirements:
- Header with status chip, elapsed, estimated remaining, overall progress bar and Cancel with confirmation.
- Contender table: tier badge, status chip, time vs budget mini bar, best-so-far score if available, log viewer (collapsible details element, last lines only).
- Poll every 2 seconds with setTimeout and AbortController (back off on errors, pause when document.hidden, no overlapping requests). Stop polling when the experiment is finished.
- Resume behaviour: open any experiment id from History and see the correct state, including interrupted and cancelled.
- Empty and connection-lost states.

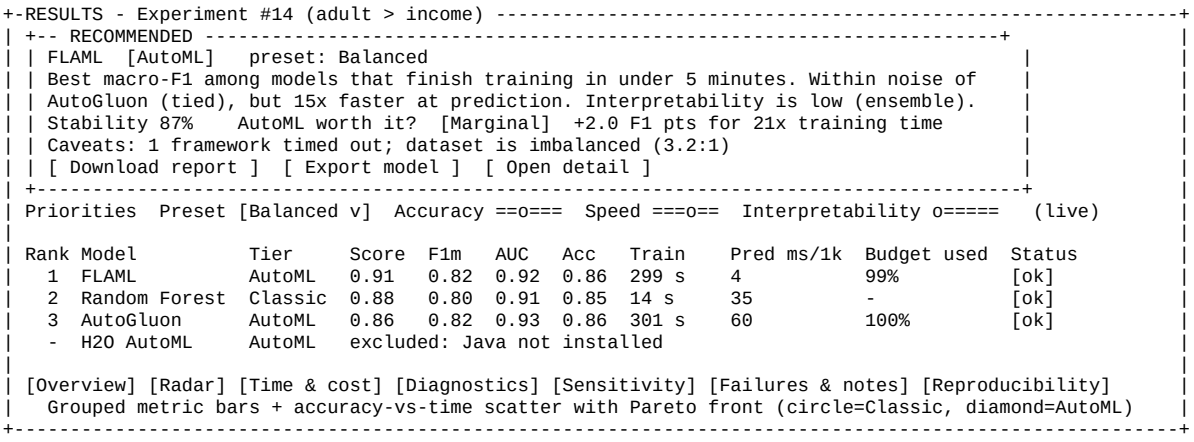
RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement and test using the FakeAdapter backend configuration (ok, slow, crash, timeout, unavailable). Report files changed and states verified.

10. Page 5: Results

Purpose: deliver the recommendation and the evidence behind it. This page is the product's reason to exist. **Priority P1.**

How it looks



How it works

- **Recommendation card** from GET /experiments/{id}/recommendation: winner, plain-language reason, runner-up and tie information, stability % from weight perturbation, and the AutoML-worth-it verdict (worth it / marginal / not worth it) with the numbers behind it.

- **Priority panel** calls POST /experiments/{id}/rerank on change. No retraining; the card, table and charts update within a second. A "Reset to run priorities" link restores the original.
- **Leaderboard:** sortable columns, tier and status filters, excluded rows greyed with the reason. Clicking a row opens Model Detail.
- **Tabs:** Overview (grouped bars and Pareto scatter); Radar (top candidates); Time and cost (training time, budget used, prediction latency on log scales, peak memory); Diagnostics (confusion matrix and ROC for a selectable model); Sensitivity (rank shifts under weight changes); Failures and notes (every failed or skipped candidate with reason and the dataset warnings); Reproducibility (seed, library versions, hardware, config).
- **Honesty rules:** if the top two are statistically tied, the card says so. If results are hold-out only (no repeated folds), the caption says that differences below a few points may be noise.

States

- **Run still in progress:** banner with link to Run Monitor and partial results. **No model finished:** error state listing why. **Only classic finished:** normal view with a note that AutoML frameworks failed and a link to the reasons.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the Results page (web/results.html and web/js/pages/results.js), the most important page of the product.

Requirements:
- Recommendation card at the top using the shared component: winner, explanation, tie information, stability percentage, AutoML verdict with numbers, caveats, and action buttons (report, export model, open detail).
- Priority panel with live re-ranking through /rerank; show a small spinner, never a full page reload; keep a reset link.
- Leaderboard with tier badge, status chip, sorting, filters, greyed excluded rows with reasons, click through to Model Detail (pass experiment id and model id).
- Seven tabs as in the blueprint, built with the shared Chart frame and the Plotly theme. Add a data-table fallback for each chart.
- All empty, partial, error and success states listed in the blueprint.

RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets. Do not compute rankings, normalisation or statistics in the UI.

OUTPUT:
Implement and test with a finished experiment that contains classic and AutoML rows, a failed row and an excluded row. Report files changed, endpoints used, and checks done.

11. Page 6: Model Detail

Purpose: inspect one candidate in depth and export it. Opened from Results. **Priority P2.**

How it looks

```
+-MODEL DETAIL - FLAML [AutoML]   rank 1 of 10   [ok]        < Back to results    Compare with [ v ] ----+
| F1 macro 0.82    ROC-AUC 0.92    Accuracy 0.86    Precision 0.80    Recall 0.84                        |
| (hold-out; Classic models also show mean +- std over 5 folds)                                          |
|                                                                                                        |
| [Performance] [Configuration] [Internals] [Time & resources] [Export]                                  |
|                                                                                                        |
| Confusion matrix (normalised [x])            Per-class table        Precision Recall F1               |
|           pred 0   pred 1                     class 0                0.89      0.92   0.90            |
|   true 0   0.92     0.08                      class 1                0.80      0.84   0.82            |
|   true 1   0.16     0.84                     ROC curve (binary) / one-vs-rest (multiclass)            |
+--------------------------------------------------------------------------------------------------------+
```

How it works

- **Performance:** confusion matrix with a normalise toggle, per-class precision, recall and F1, ROC curve (binary) or one-vs-rest curves (multiclass).
- **Configuration:** classic models show the best hyper-parameters and tuning method; AutoML shows framework version, preset or mode, optimisation metric used, budget, cores.
- **Internals:** classic models show feature importance when the model supports it; AutoML shows the framework's own internal leaderboard and model summary. If not available, the tab says why.
- **Time and resources:** training time, budget used, prediction latency per 1,000 rows, peak memory.
- **Export:** classic = one joblib bundle (preprocessor, model, class labels, library versions, usage snippet); AutoML = the framework's native artifact in a zip with a warning that the same framework version is needed to load it.
- **Compare with:** pick a second model to overlay confusion matrix and ROC curves.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the Model Detail page (web/model-detail.html and web/js/pages/model-detail.js), opened from Results with an experiment id and a model id.

Requirements:
- Header with tier badge, status chip, rank, back link and Compare-with dropdown.
- Metric tiles; five tabs as in the blueprint with graceful messages when a part is not available for that model (for example no feature importance, or hold-out only for AutoML).
- Export tab calling GET /experiments/{id}/export?model=... with the version warning for AutoML artifacts.

- Direct-link support: opening the page with ids in the query string works after a refresh.

RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement and test with one classic model, one AutoML model and one failed model. Report files changed and gaps.

12. Page 7: History

Purpose: find past experiments, rerun them, and compare two or three. **Priority P2.**

How it looks

```
+-HISTORY ------------------------------------------------------------------------------------------+
| Search [_____________]  Dataset [all v]  Status [all v]  Winner [all v]   [ Compare selected (2) ]  |
|                                                                                                    |
| [x]  #14  adult        26 Oct  11  FLAML          Balanced   [ok] done         [Open] [Rerun] [Delete] |
| [x]  #13  adult        25 Oct  6   Random Forest  Balanced   [ok] done         [Open] [Rerun] [Delete] |
| [ ]  #12  stroke       24 Oct  11  -              Balanced   [x] failed        [Open] [Rerun] [Delete] |
|                                                                                                    |
| COMPARE #14 vs #13                                                                                 |
| (!) Same dataset. Budget differs (300 s vs 60 s), so differences may come from budget.             |
| Metric          #14 FLAML    #13 RandomForest   Delta        Config differences                    |
| F1 macro        0.82         0.80               +0.02        budget, contenders                    |
| Training time   299 s        14 s               21x          library versions same                 |
+----------------------------------------------------------------------------------------------------+
```

How it works

- List from GET /experiments with search and filters. Open goes to Results (or Run Monitor if unfinished). Rerun creates a new experiment with the same configuration and a new id (never overwrites).
- Compare accepts 2 to 3 experiments. A banner explains whether results are comparable: different dataset, hardware, library versions or budgets are called out explicitly.
- Deleting asks for confirmation and shows what will be removed.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the History page (web/history.html and web/js/pages/history.js) with list, filters, rerun, delete and compare.

Requirements:

- Filterable table with selection checkboxes, actions per row, status chips and winner with tier badge.
- Compare view for 2 to 3 experiments: winner cards side by side, metric delta table, configuration differences highlighted, and a comparability banner generated from the stored hardware, version and budget data.
- Rerun using the stored config; delete with confirmation; empty state.

RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets.

OUTPUT:
Implement and test compare with same-dataset, different-dataset and different-budget cases. Report files changed and issues.

13. Page 8: Frameworks

Purpose: show which AutoML frameworks work on this machine and let the user set defaults. **Priority P2.**

How it looks

```
+-FRAMEWORKS ------------------------------------------------------------------------------------------+
| Machine: 8 cores, 16 GB RAM, Linux (example)          Recommended limits: 4 cores per run, 8 GB       |
|                                                                                                       |
| Framework       Strategy                   Package        Version  Status                Action       |
| AutoGluon       Stacked ensembles          autogluon      x.y      [ok] ready             [Smoke test] |
| Auto-sklearn    Bayesian opt. of pipelines auto-sklearn   x.y      [x] unsupported: Linux [How to fix] |
| Auto-sklearn 2  Bayesian opt. iterative    auto-sklearn   x.y      [x] unsupported: Linux [How to fix] |
| FLAML           Cost-frugal optimisation   flaml          x.y      [ok] ready             [Smoke test] |
| H2O AutoML      Random search+ensembling   h2o            x.y      [-] unavailable: Java  [How to fix] |
| (GAMA, LightAutoML, MLJAR, NaiveAutoML, TPOT: same columns)                                           |
|                                                                                                       |
| DEFAULTS  Budget [300 s]  Cores [4]  Memory [8 GB]  Grace [60 s]  Max upload [200 MB]  [ Save ]       |
| KNOWN LIMITATIONS  (from the compatibility report: what could not run and why)                        |
+-------------------------------------------------------------------------------------------------------+
```

How it works

- Reads GET /frameworks: installed version, environment status, platform support and the reason when not ready. Status comes from the live check, not from stored data.
- **Smoke test** runs a 30 to 60 second tiny fit through the same runner and shows pass or fail with the error text.
- **How to fix** shows the exact command to copy (for example creating the isolated environment). The UI never installs software itself; running installs from a browser button is a security risk and is deliberately left out.
- Defaults are saved through PUT /settings and are used to pre-fill New Benchmark.

Prompt – paste into the IDE agent
You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.

CONTEXT:
Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.

TASK:
Build the Frameworks page (web/frameworks.html and web/js/pages/frameworks.js).

Requirements:
- Machine summary (cores, RAM, OS) with recommended limits.
- Framework table with strategy one-liner (from configs/frameworks_info.yaml), version, status chip with reason, Smoke test button and How to fix collapsible section with a copyable command.
- Defaults form (budget, cores, memory, grace, upload limit) saved through the settings endpoint.
- Known limitations table generated from the compatibility report file if present.

RULES:
The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets. The UI must never execute install commands.

OUTPUT:
Implement and test with a mix of ready, unavailable and unsupported frameworks. Report files changed and endpoints used.

14. Page 9: Methodology

Purpose: let reviewers and users see exactly how numbers are produced and what the limits are. Static content. **Priority P3.**

How it looks

```
+-METHODOLOGY ------------------------------------------------------------------------------------+
| Search [____________]                                                                            |
|                                                                                                  |
| v How a benchmark runs      split > Classic and AutoML tiers > central scoring > ranking > pick  |
| v Fair-comparison protocol  same split, seed, cores, memory; one framework at a time; raw data   |
| v Metric glossary           what each metric means and when to trust it                          |
| v How the recommendation works   weights, normalisation, constraints, TOPSIS, Pareto front, stability |
| v The AutoML frameworks    one row each: strategy, strengths, known limits (BO, EO, RS, CFO explained) |
| v Limitations               budget-driven rankings, hardware, version ageing, platform limits    |
| v References              AMLB and the papers behind this project                                |
+------------------------------------------------------------------------------------------------+
```

How it works

- Collapsible details sections for each topic; content in Markdown files under web/content/ so it can be edited without touching code (rendered safely, no raw HTML).
- Links from chart captions on Results and from the metric labels on New Benchmark jump to the matching section.
- Shows the protocol values actually in use (default budget, seed, folds) read from the configuration, so documentation cannot drift from the code.

Prompt – paste into the IDE agent

> You are a senior full-stack engineer and UI/UX-minded developer working inside an existing production codebase.
>
> CONTEXT:
> Read AGENTS.md, Sections 2 to 4 of this blueprint (look and feel, layout, shared components), ARCHITECTURE.md and API.md before changing anything. The FastAPI backend exists and a Streamlit prototype exists only as a reference; the new frontend is plain HTML, CSS and JavaScript under web/. Inspect web/, app/ and src/acbs/api/ first and reuse what is there.
>
> TASK:
> Build the Methodology page (web/methodology.html and web/js/pages/methodology.js) with content loaded from Markdown files in web/content/.
>
> Requirements:
> - Seven collapsible sections as in the blueprint and a simple text search.
> - Metric glossary and framework strategy table written from the project documents (AMLB and the course literature review); do not invent claims or numbers.
> - Protocol values read live from configs/default.yaml.
> - Anchor links usable from other pages.
>
> RULES:
> The UI talks only to the JSON API under /api and contains no ML logic. Use plain HTML, CSS and vanilla JavaScript ES modules with no build step and no framework. Use CSS tokens, never inline styles or inline scripts; never put untrusted text into innerHTML (use textContent). Reuse the shared components from web/js/components/. Handle loading, empty, error and success states. Do not invent features or endpoints silently: if a needed endpoint is missing, list it, implement the smallest version following API.md, and report it. Keep changes focused and never expose secrets. Keep claims factual and cite the source document for each statement.
>
> OUTPUT:
> Implement and report files changed.

15. Build Order and Final Acceptance

Order	Build	Why this order
1	Design foundation and shared components (Section 4)	Everything else reuses it
2	Datasets, then New Benchmark	You cannot run anything without data and a config
3	Run Monitor, then Results	Completes the core journey: upload, run, recommendation
4	Model Detail, History, Frameworks	Depth and operations
5	Home, Methodology	Polish and credibility

Final UI acceptance checklist

- A new user reaches a recommendation in 5 clicks without reading documentation.
- Every page shows correct loading, empty, error and success states.
- Tier and status are distinguishable without colour.
- A failed or unavailable framework is always visible with its reason.
- Changing priorities updates the recommendation without retraining.
- The page works at 1024 px width and by keyboard only.

- No page contains ML logic, secrets, or hard-coded results.

Prompt – paste into the IDE agent
You are a senior engineering lead approving a UI for release.

CONTEXT:
Audit the finished HTML/CSS/JS frontend against Sections 1 to 14 of this blueprint, ARCHITECTURE.md, API.md and AGENTS.md.

TASK:
Perform a final end-to-end UI audit.

Requirements:
- Walk the full journey on a small dataset with classic models, then with two AutoML frameworks, and once with a deliberately failing framework.
- Verify every page state, the acceptance checklist, charts against their data tables, and that /rerank never retrains.
- Check keyboard navigation, contrast of status chips, layout at 1024 px width, the Content-Security-Policy (no console violations), and run the Playwright end-to-end journey.

RULES:
Do not assume something works because the code exists; run it. Prioritise blockers. Do not make risky changes automatically.

OUTPUT:
Give a verdict: READY, READY WITH WARNINGS, or NOT READY. List findings by severity, required fixes, tests performed, and anything that must be checked manually.

16. Decisions That Need Your Approval

Decision	My recommendation	Alternative and cost
UI technology	Plain HTML, CSS and JavaScript (no build step), multi-page, served by FastAPI at /.	React or Vue add components and routing but need a build toolchain; Streamlit is faster to build but limits layout and polling control.
Login	No login: single-user, local tool.	Add authentication only if you host it for several users.
Installing frameworks from the UI	No. Show copyable commands only.	In-app install is convenient but a security risk.
Page count	9 pages as listed.	Merge Home into Datasets and Methodology into Help links to cut to 7 if time is short.
Dark theme	Tokens support it; not hand-tuned in the first release.	Full dark design adds testing on every chart.
Mobile layout	Not a goal.	Responsive phone layout needs a redesign of tables and charts.